Інструкція до практичної роботи №97

ТЕМА: "ОБ'ЄКТНО-ОРІЄНТОВАНЕ ПРОГРАМУВАННЯ У C++ BUILDER"

Створення власних класів. Наслідування і віртуальні функції. Інтерпретатор математичних виразів.

Мета роботи. Навчитися створювати та використовувати власні класи, нащадки, віртуальні функції, програмно реалізовувати деревоподібні графи.

Розробити інтерпретатор для обчислення значення математичних виразів, заданих як рядки символів. У виразах обмежитись операціями додавання і множення, які застосовуються до операндів чисел і змінних.

Теоретичні відомості й аналіз задачі.

Розглянемо вираз $x^2 + 2x + 4$. Цей вираз може бути зображений у вигляді деревоподібної структури - графа (див. рис. 16 a). Для обчислення виразу, поданого графом, наприклад для x=1, необхідно, починаючи з нижнього рівня, обчислювати значення у вершинах і поступово підніматися до кореня (див. рис. 16 b-d).

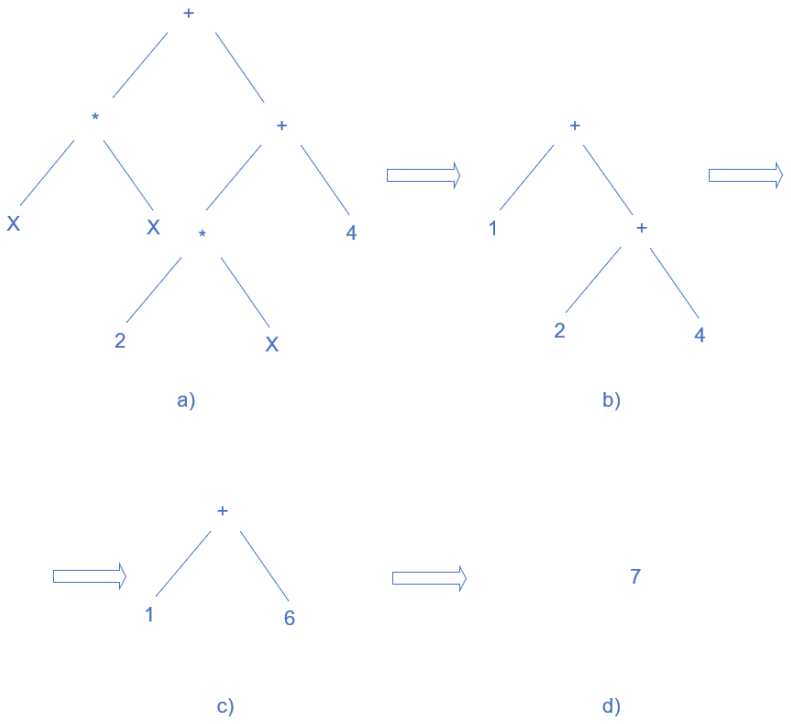


Рис. 16. Зображення арифметичного виразу у вигляді графа-дерева

Зведемо задачу до двох підзадач:

- побудова дерева за заданим виразом;
- обчислення значення виразу згідно з побудованим деревом.

Для реалізації першої підзадачі опишемо новий тип даних - базовий (батьківський) клас „вершина дерева". Назвемо цей тип Telement — тип „елемент дерева". Такий базовий тип володітиме спільними для усіх вершин дерева властивостями, а саме: вказівниками left і right на ліву та праву вершини підпорядкованого дерева (типу Telement*) і функцією rezult, яка обчислює результат у цій вершині:

```
class Telement {
protected:
//Вказівники на ліву і праву гілки піддерева
```

```
    Telement  *left, *right;
    // Конструктор - створює об'єкт за вказівниками
    Telement (Telement* L,Telement* r)// на ліву і праву вершини
    {
left = L; right = r;
}
    public:
    ~Telement(void)            // Деструктор - знищує вказівники на ліву і праву
    {delete left; delete right;} // вершини піддерева
     virtual float rezult(void) {}
        };
```

Специфікатор доступу protected для вказівників left і right вказує на те, що ззовні свого класу вони є недоступними, але видимі для нащадків цього класу. Для конструктора Telement цей специфікатор означає, що він може бути виконаний, з конструктора класу-нащадка.

Розглянемо деструктор -Telement. Як відомо, деструктор викликається для знищення об’єкта (наприклад, командою delete). Під час виклику ~Telement() для вершини дерева буде зініційоване виконання цього деструктора для вершин лівого і правого піддерев. Отже, шляхом рекурсивного виклику деструктора буде знищено ціле дерево.

Зазначимо, що функція результату rezult() повинна обчислювати і повертати значення дерева математичного виразу. Ця функція буде в усіх класах, але її реалізації залежатимуть від конкретного нащадка і будуть відрізнятися. Тому у предку вона є нереалізованою. Крім того, висновок про те, функція якого із похідних класів повинна бути викликана, приймається лише на етапі виконання програми після побудови дерева конкретного математичного виразу (це і є пізнє, або динамічне зв’язування). Нагадаємо, що такі функції називаються віртуальними і позначаються ключовим словом virtual.

Вершиною (елементом) побудованого дерева може бути операція „+”, „*” або число. Тому успадкуємо від класу Telement (див. рис. 17) три класи-нащадки: два для арифметичних дій („+” - Plus, „*” - Mult) та для числа (Number).

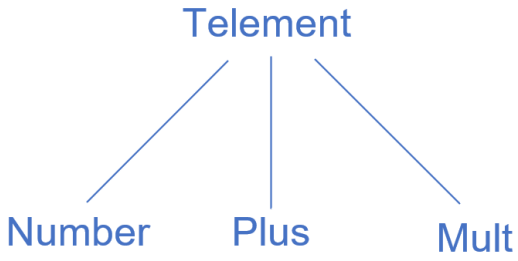


Рис. 17. Ієрархічне дерево класів

Розглядаючи дерево математичного виразу на рис. 16 а, та роблячи узагальнення, сформулюємо:

Правило 1. Вершини „число” та „змінна” (клас Number) завжди знаходяться на останньому рівні і не мають піддерев. Тому в цьому класі вказівники на ліву і праву підпорядковані вершини будуть нульовими (NULL), а функція результату rezult() повертає константу чи значення змінної.

Правило 2. Об’єкти класів арифметичних дій („+” та „*”) завжди мають як ліву, так і праву гілки. Для класів Plus і Mult двома гілками піддерева будуть вказівники на об’єкти класу Telement. Функція результату цих класів має виконати функції rezult() для лівої та правої підпорядкованих вершин, а також операції додавання чи множення. Тому виклики

функцій rezult() для підпорядкованих вершин спричинять рекурсивні виклики функцій результату до низу дерева.

Важливо зазначити, що оскільки підпорядковані вершини об’єкта класу Plus чи Mult є однотипними (базового типу Telement), стають правомірними виклики функцій rezult() цих вершин незалежно від їх конкретного типу. Зважаючи на те, що у базовому класі функція результату описана як віртуальна, механізм пізнього (динамічного) зв’язування забезпечує виклик успадкованої функції rezult() вершини потрібного класу, а не порожню заготовку функції Telement::rezult(). У вищенаведе ній задачі ми вимушені використовувати механізм пізнього зв’язування, оскільки математичний вираз задається на етапі виконання програми в режимі діалогу, і структура дерева на момент опису створюваних класів невідома.

Існування базового класу Telement дає змогу використовувати єдиний для всіх побудованих класів описаний вище деструктор ~Telement(): завдяки однотипності вершин знищуються об’єкти класів Plus, Mult та Number.

Опишемо класи-нащадки. Почнемо з класу Number:

зв’язування, оскільки математичний вираз задається на етапі виконання програми в режимі діалогу, і структура дерева на момент опису створюваних класів невідома.

Існування базового класу Telement дає змогу використовувати єдиний для всіх побудованих класів описаний вище деструктор ~Telement(): завдяки однотипності вершин знищуються об’єкти класів Plus, Mult та Number.

Опишемо класи-нащадки. Почнемо з класу Number:

```
class Number: public Telement
 {
  float f;              // дійсне число знаходиться в приватній змінній f public:
  Number(float F):Telement(NULL , NULL)
           {             //  конструктор - присвоює числу f певне значення, а
             f = F;          // вказівники на підпорядковані вершини занулені
           }
float rezult(void)
   {return f;}               //функція       результату повертає число f
  };
```

Об’єкт класу Number характеризує число, яке повинно міститися у конкретній вершині. Відповідно до правила 1, цей клас піддерев не має. Тому під час виклику конструктора базового класу замість вказівників на ліву і праву гілку передаються NULL-вказівники.

Функція rezult() повертає значення числа. Ще раз звернемо увагу на кваліфікатор virtual в описі цієї функції у базовому класі. Він означає: якщо доступ до об’єкта похідного класу Number виконуватиметься через вказівник на базовий клас, то викликатиметься функція rezult() похідного, а не базового класу. Проілюструємо це на прикладі:

```
Telement* A = new Number(3);
// Тут A - вказівник на тип Telement, *A - об'єкт класу Number
A -> rezult();
// Викликається метод result() класу Number, а не класу Telement.
// Тому повертається число 3, а не виконується порожня заготовка
```

// віртуального методу result() класу Telement.

У цьому фрагменті виконується виклик функції Number::rezult(). Якщо б у базовому класі в описі функції rezult() не було слова virtual, то в цьому прикладі викликалась би функція Telement::rezult().

Опишемо класи, що реалізують операції. Опиратимемося на сформульовані вище правила.

```
struct Plus: public Telement{
    Plus(Telement* L, Telement* r):Telement(L, r) // конструктор - викликає
                    {};                                  //  конструктор базового класу
    float rezult(void)                          // Результатом є сума результатів лівої і правої
     {return left->rezult() + right->rezult();}     //підпорядкованих вершин
};
    class Plus: public Telement{ // Альтернативний опис класу Plus public:
Plus ( Telement* L, Telement* r): Telement (L, r) {};
      float rezult(void)
      {
      return left -> rezult() + right -> rezult();
      }
};
```

Конструктор цього класу лише викликає конструктор базового класу згідно з першим правилом. Функція rezult() обчислює значення на лівій і правій гілках, а тоді їх підсумовує. Аналогічно визначена функція rezult() для класу Mult (результати на підпорядкованих ребрах перемножуються):

```
struct Mult: public Telement{
   Mult(Telement* L, Telement* r):Telement(L, r) {};
   float rezult(void)
  {
    return left -> rezult() * right -> rezult();
  }
};
```

Якщо гілки є піддеревами, то виконається виклик функції rezult() для вершини кожного піддерева, і так аж до останнього рівня, де rezult() повертає значення числа чи змінної. Тому достатньо викликати rezult() для кореня, після чого буде обчислено значення виразу для всього дерева.

Приклад 1. Створимо дерево виразу 3*4 + 2:

```
Number a(3), b(4),c(2);// Об'єкти класу Number: a =3, b = 4 і c = 2
Mult m(&a, &b);                  //  a*b
Plus p(&m, &c);        // m + c
```

Викликавши функції rezult() одержимо такі результати:

```
p.rezult();          //повертаєзначення 14
m.rezult();          //дорівнює 12
a.rezult();          //повертаєзначення 3
```

Отже, ми отримали набір класів, об’єктами яких зручно реалізувати дерево, побудоване на базі математичного виразу. Для завершення реалізації першої під задачі

розглянемо функцію, яка будує таке дерево за заданим у вигляді рядка символів математичним виразом, у якому змінна x позначається малою латинською буквою, значення змінної x знаходиться у полі редагування Edit3 (див. рис. 18, 19). Функція form() формує дерево внаслідок рекурсивних викликів. Спочатку функція виділяє доданки, далі кожен з доданків розкладає на множники, потім обробляє змінні та числа.

```
Telement* form(AnsiString s)  // s - заданий рядок символів
{          // Результат - вказівник на вершину дерева побудованого виразу
    Telement* h;
    int p;                    // p- позиція оператора „+" чи “*”
    int L = s.Length(); // L - довжина заданого рядка символів
    AnsiString s1, s2;  // шукані операнди вершини „+” чи „*”
    if ((p = s.Pos(“+”)) >1)        // Якщо в рядку є операція „+"
      {
       s1 = s.SubString(1, p - 1); //        знаходимо підрядки s1 і s2, -
       s2 = s.SubString(p + 1,1 - p); // операнди операції „+”
       h = new Plus(form(s1), form(s2));
       // Створюємо об’єкт Plus з  підвершинами - знайденими операндами операції
       //які аналізуватимуться рекурсивними викликами функції form
       }
else if ((p = s.Pos("*“)) > 1)//Аналогічно розбиваємо     рядок на
                                          // операцію „*” і два операнди s1 і s2
    {
      s1 = s.SubString(1, p -1);
      s2 = s.SubString(p + 1,1 - p);
      h = new Mult(form(s1), form(s2)); // Створюємо об’єкт Multi
                                        // рекурсивно викликаємо функцію form для знайдених
операндів
    else if (s == “x”) // Якщо рядок є змінною x: створюємо об'єкт
                       // „число" й  ініціалізуємо його числовим значенням змінної x з поля Edit3:
           h = new Number(StrToFloat(Form1->Edit3->Text));
           else
             // Якщо вираз задано правильно, в результаті відсіювання
             // операторів і змінних „x” в рядку s залишається
             // константа. Створюємо об’єкт „число” і ініціалізуємо
              // його числовим значенням рядка s:
             h = new Number(StrToFloat(s));
             return h; // Повертаємо вказівник на створений об’єкт
     }
```

Висновок. За допомогою засобів ООП створено зручний для використання та розширення набір програмних засобів, які розв’язують задачу побудови інтерпретатора математичних виразів. Продемонстровані прийоми використання класів застосовуються для побудови й опрацювання таких структур даних, як дерева, стеки, черги, динамічні списки та ін.

Варіант і вимоги до здачі

а) N — ваш порядковий номер у списку групи (журнал). Дані завдань залежать від N

(mod — остача від ділення):

Завдання	Дані варіанта	Приклад N = 7
1	вираз a*b + c: a = N mod 5 + 2, b = N mod 4 + 3, c = N mod 3 + 1	4*6 + 2
2	f(x, y) = A·x·y + B·x + C·y + D; A = N mod 3 + 1, B = N mod 4 + 1, C = N mod 5 + 1, D = N; x = 2, y = 3	2*x*y + 4*x + 3*y + 7

б) Здайте архівом папку проєкту Прізвище_ПР97 (файли *.bpr, *.cpp, *.h, *.dfm, *.res та файли даних, потрібні для запуску (зображення, звуки)) і документ Word зі звітом; виконуваний файл *.exe додавати не потрібно.

в) До звіту додайте скріншоти форми в режимі проєктування й під час роботи програми (з введеними даними) та текст змінених вами обробників подій.

г) Класи Telement, Number, Plus, Mult і функцію form() введіть, як у теорії; нові класи й змінні оголошуйте в тому самому модулі; точність — два знаки після коми (ComboBox1.Text = 2).

д) У коді залиште короткі коментарі власними словами перед основними блоками.

е) Будьте готові пояснити будь-який рядок свого коду й внести в нього невелику зміну (див. розділ «Захист роботи»).

Хід роботи

1. Завантажте C++ Builder.
2. Замініть заголовок (Caption) форми з “Forml” на “Обчислення виразу”.
3. Роботу з програмою завершувати натисканням на кнопку "Вихід”.
4. Розташуйте на формі об’єкти так, як показано на рис. 18.
5. Уведіть описи класів і функцій, які використовуватимуться у цій задачі.

Для цього відкрийте вікно тексту програми і після рядка
TForml :Form1; // Цей рядок уже є у програмі
вставте програмний код опису класів Telement, Number, Plus, Mult і текст функції формування дерева form (див. „Теоретичні відомості й аналіз задачі”). Збережіть проект та текст програми.

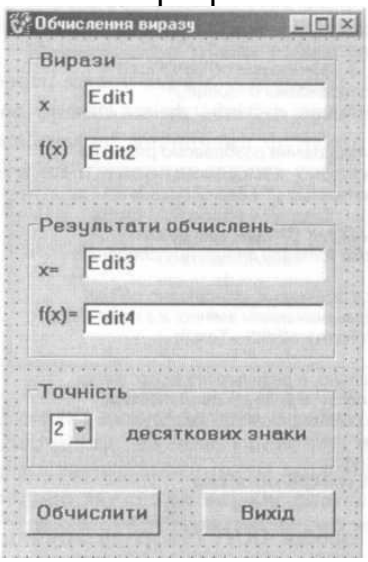


Рис. 18. Форма "Обчислення виразу"

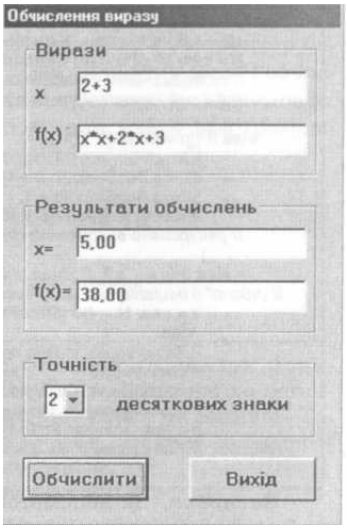


Рис. 19. Приклад роботи програми

6. Уведіть математичні вирази у поля Editl і Edit2 та очистіть поля результатів Edit3 та Edit4. Запрограмуйте кнопку „Вихід”.

Зверніть увагу на те, що у виразах допускаються лише операції „+”, „*”, дійсні числа та змінна x, задана малою латинською літерою. У виразі для x не використовуйте змінну x.

Для набору символа десяткової крапки у дійсних числах використайте символ, передбачений операційною системою комп'ютера. Тестовий приклад і результати обчислень показані на рис. 19.

7. Задайте початкову точність обчислень, що дорівнює двом десятковим знакам. Для цього властивість Text об'єкта ComboBox1 задайте числом „2" без лапок.

8. Запрограмуйте кнопку „Обчислити" так:

```
void fastcall TForm1::Button1Click(TObject *Sender)
{
int dec = StrToInt(ComboBox1 -> Text); // Знаків після десят. крапки
Telement* x = form(Edit1 -> Text);          // x - вказівник на вершину
                                            // дерева, побудованого за виразом у полі Editl
                                  // Результат обчислення x записуємо як рядок символів у полі
Edit3 Edit3 -> Text = FloatToStrF(x -> rezult(), ffFixed, 25, dec);
                                // fx - вказівник на вершину дерева,  побудованого за виразом у Edit2
 Telement*  fx = form(Edit2 -> Text);
                                 // Результат обчислення f(x) записуємо як рядок символів у Edit4
Edit4 -> Text = FloatToStrF(fx -> rezult(), ffFixed, 25, dec);
}
```

9. Запрограмуйте функцію, яка виконуватиметься після зміни числа у полі ComboBoxl. Для цього запрограмуйте подію Change об'єкта ComboBoxl так, щоб виконувались дії для кнопки „Обчислити" (функція ButtonlClick):

```
void fastcall TForml ::ComboBox1Change(TObject 'Sender)
{
Buttonl Click(Button1);
// Допустимі також записи: Button 1 CHck(ComboBox 1); Button 1 Ciick(NULL);
}
```

Зауваження 1. *Фактичним параметром функції Buttonl Click може бути вказівник на будь-який візуальний об'єкт форми, адже в описі цієї функції формальним параметром є вказівник на базовий тип TObject середовища візуального програмування C++ Builder (див. п.8). Таким аргументом може бути також нуль-вказівник NULL. У складніших програмах цей параметр можна використовувати для передачі певної інформації у функцію.*

Зауваження 2. *Повторні виклики функції Buttonl Click() після кожного виконання події ComboBox1 Change щоразу приводити- муть до створення дерев математичних виразів для x та f(x). Адже вказівники на ці дерева є локальними змінними функції ButtonlClick(). Тому побудовані дерева знищуватимуться після завершення роботи цієї функції шляхом неявного виклику деструкторів об'єктів x та fx:*

x -> ~Telement(); fx -> ~Telement();

ЗАВДАННЯ

1. Побудуйте дерево виразу 3 * 4 + 2, скориставшись прикладом із розділу „Теоретичних відомостей та аналізу задач" цього параграфа.

Приклад (N = 7): дерево виразу 4*6 + 2: p.rezult() = 26, m.rezult() = 24, a.rezult() = 4; у полях Edit — «26.00000000» (формат FloatToStrF(…, ffFixed, 25, 8)).

Підказка. Для цього вставте у форму ще одну кнопку і три додаткові поля редагування для значень функції rezult() об'єктів p, m і a. Для виведення результатів у полях Edit скористайтеся функцією FloatToStr:

Edit5->Text = FloatToStrF(p.rezult(), ffFixed, 25, 8);

2. Обчисліть значення функції від двох змінних f(x, y).

Підказка. Для цього додайте змінну y в опис функції form() та відповідне поле Edit на форму.

Приклад (N = 7): f(x, y) = 2*x*y + 4*x + 3*y + 7 при x = 2, y = 3 → «36.00»; значення x і y вводяться в окремих полях Edit.

Критерії оцінювання

Складова	**Оцінювання**	**Макс. балів**
Основна частина (кроки 1–9)	5 балів	5
Завдання 1	3 бали	3
Завдання 2	3 бали	3
Контрольні питання	відповіді у звіті	1
Разом		**12**

Кожне завдання оцінюється так: програму виконано повністю, вона працює на вимогах умови й прикладах формату виводу — 100% балів; програма працює, але є суттєве зауваження (не виконано вимогу умови, неправильне дерево виразу, неправильний формат результату) — 50%; програма не працює або не здана — 0. Контрольні питання оцінюються в 1 бал, якщо на всі питання дано правильні відповіді.

Захист роботи

Перед виставленням оцінки викладач може попросити вас пояснити будь-який рядок свого коду, показати, як змінюється значення змінних на прикладі, або внести в програму невелику зміну (наприклад, змінити вираз чи значення змінних). Якщо пояснити код не вдається, оцінку за відповідне завдання може бути знижено.

Контрольні питання

1. Що таке клас і об'єкт? Яка різниця між класом і структурою в C++?
2. Що таке наслідування й віртуальна функція?
3. Як будується деревоподібне зображення математичного виразу?
4. Для чого потрібен деструктор і як він викликається в цій програмі?

Відповідайте на контрольні питання письмово у звіті (документ Word разом зі скріншотами): 1–2 речення на кожне питання, власними словами.